UNIDAD 8

Trabajo práctico 08 - Endpoints persistentes de reservas de salas

1. Modalidad y entrega

- Individual, por URL de GitHub.
- Vencimiento: lunes siguiente a las 19:59.
- Punto de partida: copia funcional del TP 07 individual.
- Nombre sugerido: `tp-08-salas-http-mongodb` .
- Entorno: MongoDB Atlas Free (M0), base `reservas_salas_u8` , colección `reservas` .
- Pruebas de la API: Postman de escritorio, siguiendo el procedimiento de la clase 1.

Conservar el TP anterior. No copiar historial Git, node_modules ni .env. Mantener versiones, manifiesto y lockfile. Si el TP 07 solo llegó a conexión, completar su modelo y operaciones antes de integrar HTTP.

2. Consigna

Migrar creación, listado y consulta de reservas a MongoDB mediante un servicio asíncrono, conservando el sitio HTML y agregando una API JSON. El sitio y la API deben recuperar los mismos documentos.

La entrega ya no puede depender de un arreglo o de cargar la semilla del JSON al reiniciar. Mantener esa referencia si resulta útil, pero retirar sus importaciones del arranque persistente. No borrar datos ni insertar automáticamente al iniciar o al listar.

3. Estructura y arranque

Mantener src/config, modelos, servicios, controladores, rutas, middleware y scripts en la raíz. Separar router HTML y router API; no escribir rutas HTTP dentro del modelo ni respuestas dentro del servicio.

El arranque lee entorno, espera la conexión y luego escucha el puerto. Conservar la corrección DNS acordada con la red, sin duplicarla por solicitud. Los programas de terminal cierran al terminar; el servidor no desconecta después de cada petición. Incluir cierre ordenado ante SIGINT y SIGTERM, como en clase.

Documentar comando de inicio con `--env-file=.env` o script equivalente. El .env no se entrega. La plantilla incluye PORT, NODE_ENV y una URI de Atlas con marcadores y base reservas_salas_u8. No prometer que un nombre HOST ilustrativo permite conectar.

4. Modelo y entrada admitida

Conservar los seis campos del TP 07:

Campo	Regla
estudiante	String obligatorio, trim
email	String obligatorio, trim, presencia de @
sala	String obligatorio, enum Sala Norte / Sala Sur / Sala Multimedia
fecha	String obligatorio, trim
turno	String obligatorio, enum Mañana / Tarde / Noche
personas	Number obligatorio, entero entre 1 y 6

Agregar `estado` con default `pendiente` y opciones de esquema pendiente/confirmada. El cliente **no lo asigna en la creación**; la API de esta unidad no incluye confirmar una reserva. El default no constituye autenticación, autorización ni migración de documentos viejos.

Copiar explícitamente solo los seis campos admitidos al crear. No pasar req.body completo a Mongoose. Rechazar cuerpos ausentes, texto sin parser, tipos numéricos booleanos, arreglos u objetos y entradas vacías. El formulario conserva valores y 400. Mantener las validaciones del modelo, no solo las del middleware.

Estas reglas no resuelven email completo, fecha real o futura, solapamientos, cupos compartidos ni duplicados. No se exige índice único.

5. Contrato mínimo de la API

Método y dirección	Resultado
GET /api/reservas	200 y arreglo
GET /api/reservas/:id	200 y recurso
POST /api/reservas	201, recurso guardado y Location
GET /api/reservas?sala=Sala%20Norte	200 y coincidencias exactas
ID mal formado	400 JSON
ID bien formado inexistente	404 JSON
Creación inválida o JSON mal formado	400 JSON, sin escritura
Fallo inesperado durante operación	500 JSON, sin traza ni secreto

La representación contiene `id` textual derivado del ObjectId y los campos del recurso. No aplicar Number al parámetro. Consultar la Location de un POST debe recuperar ese recurso. Una lista sin coincidencias devuelve [], no 404.

El filtro solo acepta una sala textual; armar el objeto de criterio explícitamente. No pasar req.query completo ni aceptar operadores MongoDB enviados por el cliente. Si el parámetro aparece repetido como arreglo, responder 400. No se exige paginar ni ordenar.

No se implementan PUT, PATCH o DELETE en esta entrega. No se publica este servidor en Internet sin las protecciones que se aprenderán después.

Probar la API con Postman

Iniciar el servidor con el comando documentado en el README y mantenerlo activo mientras se envían las solicitudes. Las direcciones siguientes usan el puerto 3000; adaptarlo si se configuró otro puerto.

Consultar el listado inicial

En Postman, preparar una solicitud **GET** a `http://localhost:3000/api/reservas` , con **Body en none**. Pulsar **Send** y registrar **200**, el arreglo recibido y la cantidad inicial. Si la base es nueva, un arreglo vacío es válido.

Crear una reserva

Preparar una solicitud **POST** a `http://localhost:3000/api/reservas` . En **Body**, seleccionar **raw** y formato **JSON**. Usar datos ficticios como estos:

```
{
  "estudiante": "Ana Ejemplo",
  "email": "ana@example.com",
  "sala": "Sala Norte",
  "fecha": "2026-10-21",
  "turno": "Mañana",
  "personas": 2
}
```

La fecha es ilustrativa: este TP mantiene la regla de texto obligatorio y no incorpora validación de fecha futura. No enviar `estado` ni `_id` en esta creación de referencia.

Comprobar `Content-Type: application/json` en **Headers de la solicitud**. Postman lo configura al seleccionar JSON, salvo que se haya sobrescrito manualmente. Pulsar **Send una sola vez** y registrar:

- Estado **201**.
- Cuerpo con `id` textual, los seis campos y `estado: "pendiente"` .
- Cabecera `Location` en **Headers de la respuesta**.

Guardar el ID y la dirección. Otra ejecución del mismo POST válido solicita otra creación.

Consultar la reserva creada

En otra solicitud, elegir **GET**, dejar **Body en none** y utilizar la dirección recibida en `Location` . Si es relativa, agregar delante `http://localhost:3000` :

```
http://localhost:3000/api/reservas/ID_RECIBIDO
```

Reemplazar el marcador por el ID real. Registrar **200**, el mismo ID, los mismos campos y el default pendiente. Distinguir las cabeceras enviadas de las recibidas: `Location` pertenece a la respuesta del POST.

Comprobar identificadores y filtros

Realizar por separado estas solicitudes **GET**, sin cuerpo:

```
http://localhost:3000/api/reservas/1
http://localhost:3000/api/reservas/000000000000000000000000
```

La primera debe responder **400**. La segunda tiene un formato válido y debe responder **404 solo si ese documento no existe** en la colección.

Para el filtro, usar GET a `/api/reservas` y **Params** con **Key** `sala` y **Value** `Sala Norte` . Comprobar coincidencias exactas. Probar también un texto de sala sin coincidencias y verificar **200** con `[]` : el enum limita la sala al crear, mientras este filtro admite una sala textual y busca su igualdad exacta. Conservar los datos y registrar qué sala se consultó. Para probar el rechazo de parámetro repetido, activar dos filas con Key `sala` y valores distintos: debe responder **400**. Desactivar los filtros al comparar la cantidad total.

Ejecutar casos de entrada inválida

Duplicar la solicitud POST para los ensayos y cambiar un dato por vez. Probar los casos de la matriz: estudiante vacío, email sin @, sala o turno no permitido, fecha vacía y valores inválidos de personas. Para cuerpos ausentes, seleccionar **Body en none**; para JSON mal formado, enviar deliberadamente sintaxis incompleta con `Content-Type: application/json` .

Después de cada rechazo, volver al GET sin filtros y comparar cantidades. Para el caso de campos extra, enviar `estado: "confirmada"` o `_id` junto a los seis datos válidos: el servidor debe ignorarlos y crear con su propio identificador y estado pendiente. Esa prueba sí crea una reserva válida y debe registrarse como tal.

Las pruebas del formulario y las vistas se realizan en el navegador; las de la API se registran desde Postman.

6. Continuidad HTML y estado

- Inicio, CSS, layout y ruta inexistente continúan funcionando.
- Listado y detalle previos leen la base; si el sitio del TP no tenía detalle, la consulta por ID obligatoria es la de la API.
- El formulario válido espera la escritura y redirige 302.
- El formulario inválido responde 400 y conserva valores.
- /estado mantiene su contrato y obtiene el conteo desde MongoDB, no de una semilla.
- La representación preserva los nombres que necesitan las vistas.

Mostrar las modificaciones de arranque, servicio y controlador de forma coherente. No dejar un consumidor esperando un arreglo síncrono de un servicio que ahora devuelve una promesa.

7. Errores y respuestas

Con Express 5, los controladores async deben esperar o retornar sus operaciones. Registrar un middleware de errores con cuatro parámetros después de las rutas. Distinguir entrada inválida, recurso ausente y fallo interno; no transformar errores en arreglos vacíos o 200.

La respuesta API es JSON, incluso en una ruta API desconocida. La respuesta HTML no publica trazas, contraseñas ni URI. Los fallos de red/DNS al arrancar no deben anunciar que el servidor está disponible. Registrar diagnósticos sin divulgar secretos.

8. Evidencia de persistencia

1. Consultar en Postman el GET sin filtros y registrar cantidad inicial.
2. Crear una reserva por POST en Postman y guardar ID y Location.
3. Enviar un GET a la URL de Location y comparar datos y default pendiente.
4. Cerrar y volver a iniciar el servidor con la misma base.
5. Volver a enviar en Postman el GET con el mismo ID y localizar la reserva en el sitio. No repetir el POST para demostrar persistencia.
6. Listar dos veces y comprobar cantidad estable.
7. Inspeccionar reservas_salas_u8/reservas en Atlas y comparar _id.

Usar datos ficticios y capturas recortadas sin secretos. No borrar la colección ni crear nuevamente el recurso para sustituir la evidencia del reinicio.

Las evidencias de Postman deben permitir identificar método, URL, estado y resultado. Mostrar también `Location` en la creación. Registrar el mismo ID antes y después del reinicio y las cantidades de los casos de rechazo. No dar por realizada una prueba que quedó pendiente.

9. Matriz mínima

Caso	Resultado esperado
Listado vacío si la base es nueva	200, []
Creación válida	201 y Location
Consulta por ID creado	200, mismo recurso
Consulta tras reiniciar	200, mismo ID
ID "1"	400
ID válido inexistente	404
Estudiante vacío o email sin @	400, cantidad sin aumento
Sala o turno no permitido	400, cantidad sin aumento
Fecha vacía	400
Personas 0, 7, 2.5 o "abc", por separado	400
Personas true, arreglo u objeto	400
POST sin cuerpo o JSON mal formado	400 JSON
Campos extra estado o _id	No los asigna el cliente
Filtro exacto por sala	Solo coincidencias; sin coincidencias []
Sala repetida como parámetro	400
Listar dos veces	Cantidad estable
Formulario válido / inválido	302 / 400
Estado y sitio	Conteo persistente y vistas correctas
Fallo de arranque con URI ausente	No abre el puerto

Completar resultado observado y cantidades, sin inventarlos. Restaurar objetos válidos después de los ensayos. Para probar un fallo inesperado durante una operación, usar una simulación controlada en pruebas o un entorno de ejercicio acordado, sin alterar usuarios, acceso o clústeres compartidos.

10. Calidad y documentación

Ejecutar format y check sobre src y scripts, conservando las reglas de la unidad 7. Mantener CommonJS. Entregar lockfile y plantilla pública, no node_modules ni secretos.

El README debe contener origen de la copia, versiones, preparación de Atlas, entorno y DNS si se ajustó, comando de inicio, estructura por capas, tabla de endpoints, JSON de ejemplo sin secretos, matriz de resultados, prueba de reinicio, límites de validación y comparación con el TP 07.

Agregar instrucciones para reproducir las solicitudes en Postman: método, URL, Body, cabecera necesaria y respuesta esperada. Identificar las capturas o registros que respaldan los resultados observados. Puede incluirse una colección exportada con solicitudes de ejemplo sin secretos como material opcional.

Explicar qué dejó de importarse del servicio en memoria. No hace falta eliminar archivos de referencia, pero sí evitar dos fuentes activas para el mismo dominio.

11. Criterios de evaluación

Criterio	Peso
Arranque, conexión, estructura y continuidad	20%
Modelo, defaults y validación de entrada	20%
Endpoints, respuestas y representación del ID	25%
Persistencia, casos negativos y criterio exacto	25%
Calidad, README y protección de secretos	10%

12. Lista antes de entregar

- El docente puede acceder al repositorio.
- Están código, manifiesto, lockfile, plantilla y README.
- La conexión se espera antes de escuchar.
- El servidor mantiene su conexión entre solicitudes.
- API y HTML usan el mismo servicio.
- 201 y Location se imprimen después de guardar.
- Postman permite comprobar 201, Location y la consulta del mismo recurso.
- 400 y 404 no se confunden.
- Reiniciar no pierde los datos ni los duplica.
- Los rechazos no escriben.
- No hay secretos, node_modules ni datos personales reales.

Referencias de Postman

- Parámetros y cuerpo de las solicitudes
- Estructura de la respuesta